CHAPTER 13

Performance & Async

JVM · N+1 · HikariCP · Kafka · Virtual Threads · Gatling

Chapter Overview

Chapter 13 covers performance optimisation and asynchronous processing in ShopVerse: JVM profiling, DB query optimisation (N+1, EXPLAIN ANALYZE), connection pooling (HikariCP), HTTP caching headers, Spring @Async, reactive programming (WebFlux), virtual threads, Kafka async event processing, bulk operations, and load testing with Gatling.

Sub-Chapter	Topic
13-01	JVM Profiling – heap analysis, GC tuning, VisualVM
13-02	Database Query Optimisation – N+1, EXPLAIN ANALYZE, index hints
13-03	HikariCP Connection Pool – sizing formula, leak detection
13-04	HTTP Caching – Cache-Control, ETag, Last-Modified
13-05	Spring @Async – CompletableFuture, ThreadPoolTaskExecutor sizing
13-06	Spring WebFlux – Mono, Flux, reactive pipeline
13-07	Virtual Threads – Project Loom, I/O-bound throughput
13-08	Kafka Async Event Processing – decoupling write path
13-09	Bulk Operations – batch insert, JDBC batch, JPA bulk update
13-10	Pagination & Slice vs Page – keyset vs offset
13-11	Response Compression & Serialisation – gzip, Jackson tuning
13-12	Load Testing with Gatling

13-01 JVM Profiling & GC Tuning

GC Algorithm	JVM Flag	Best For	ShopVerse Recommendation
G1GC	(default Java 9+)	Balanced latency/throughput; heap 4 MB–32 GB	Default for most services
ZGC	–XX:+UseZGC	Pause <1 ms; heap up to 16 TB	Latency-sensitive order service
Shenandoah	–XX:+UseShenandoahGC	Similar to ZGC; Red Hat OpenJDK	Alternative to ZGC
SerialGC	–XX:+UseSerialGC	Single-core, small heap	Never in production

```
# JVM flags for ShopVerse product-service
-Xms512m -Xmx2g
-XX:+UseZGC # low pause GC
-XX:MaxGCPauseMillis=100 # target 100ms max pause (G1 only)
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/heapdump.hprof
-Xlog:gc*:gc.log:time,uptime,level,tags:filecount=5,filesize=20m
```

Key GC Metrics

Metric	Healthy Value	Action if High
GC pause time	G1: <200ms; ZGC: <1ms	Switch GC algorithm; increase heap
GC frequency	<5 minor/min; <1 major/hr	Investigate allocation rate; increase -Xmx
Heap utilisation	<70% after GC	Increase -Xmx or find memory leak
Allocation rate	Steady; no spikes	Profile with async-profiler; find hot allocations

N+1 Problem & Fix

```
// N+1: 1 query for orders + N queries for each order items
// BAD:
List<Order> orders = orderRepo.findByCustomerId(customerId);
orders.forEach(o -> o.getItems().size()); // triggers N lazy loads
// GOOD: JOIN FETCH in JPQL
@Query("SELECT o FROM Order o LEFT JOIN FETCH o.items WHERE o.customerId = :id")
List<Order> findByCustomerIdWithItems(@Param("id") Long id);
// GOOD: EntityGraph
@EntityGraph(attributePaths = {"items","items.product"})
List<Order> findByCustomerId(Long customerId);
```

EXPLAIN ANALYZE – PostgreSQL

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT p.* FROM products p
JOIN product_translations pt ON pt.product_id = p.id AND pt.locale = 'hi-IN'
WHERE p.category_id = 5 AND p.price BETWEEN 1000 AND 5000
ORDER BY p.ratings_avg DESC LIMIT 20;
```

EXPLAIN output node	Meaning	Action
Seq Scan on products	Full table scan – no index used	Add index on (category_id, price, ratings_avg)
Index Scan using …	Index used; check rows estimate vs actual	Check index selectivity; consider partial index
Nested Loop (cost=high)	Join strategy; can be slow for large sets	Try Hash Join via enable_nestloop=off for testing
Buffers: shared hit=0 read=N	Disk I/O occurring	Add RAM; tune shared_buffers; check index coverage

13-03 HikariCP Connection Pool

```
# application.yml
spring:
datasource:
hikari:
maximum-pool-size: 20 # = cores * 2 + spindle_count (SSD: cores*2)
minimum-idle: 5 # keep warm connections ready
connection-timeout: 3000 # fail fast if no connection available in 3s
idle-timeout: 600000 # reclaim idle connections after 10min
max-lifetime: 1800000 # recycle connections every 30min (prevent stale)
leak-detection-threshold: 10000 # warn if connection held > 10s
pool-name: ShopVerseHikariPool
```

Setting	Wrong Value Effect	Recommendation
maximum-pool-size too large	DB connection limit hit; OOM at DB	Formula: CPU*2 (SSD) or CPU*2+disk_spindles
connection-timeout too long	Request threads queue; cascading timeouts	3,000 ms – fast fail; circuit break
max-lifetime too long	DB closes connection server-side; app gets stale	1,800,000 ms (30 min) – recycle proactively
leak-detection-threshold	Slow queries hold connections; pool exhaustion	10,000 ms – log warning; find culprit query

13-04 HTTP Caching

```
// Cache-Control headers in Spring
@GetMapping("/api/v1/categories")
public ResponseEntity<List<CategoryDto>> getCategories() {
List<CategoryDto> cats = categoryService.getAll();
return ResponseEntity.ok()
.cacheControl(CacheControl.maxAge(1, TimeUnit.HOURS)
.cachePublic())
.body(cats);
}
// ETag for conditional requests
@GetMapping("/api/v1/products/{id}")
public ResponseEntity<ProductDto> getProduct(@PathVariable Long id,
WebRequest req) {
ProductDto dto = productService.findById(id);
String eTag = "\"" + dto.getVersion() + "\"";
if (req.checkNotModified(eTag)) return null; // 304 Not Modified
return ResponseEntity.ok().eTag(eTag).body(dto);
}
```

Header	Value	Effect
Cache-Control: max-age=3600	Cache for 1 hour	Client/CDN caches response; no server call
Cache-Control: no-cache	Always revalidate	Conditional GET (ETag/Last-Modified) required
ETag: "42"	Resource version fingerprint	304 Not Modified if ETag matches; saves bandwidth
Vary: Accept-Language	Separate cache per locale	CDN caches en-IN and hi-IN responses separately

13-05 Spring @Async & Executor Sizing

```
@Configuration @EnableAsync
public class AsyncConfig {
@Bean("emailExecutor")
public Executor emailExecutor() {
ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
e.setCorePoolSize(5);
e.setMaxPoolSize(20);
e.setQueueCapacity(1000);
e.setKeepAliveSeconds(60);
e.setRejectedExecutionHandler(new CallerRunsPolicy());
e.setThreadNamePrefix("email-");
e.initialize();
return e;
}
}
@Async("emailExecutor")
public CompletableFuture<Void> sendOrderConfirmation(Order order) {
emailService.sendHtml(order.getEmail(), template(order));
return CompletableFuture.completedFuture(null);
}
```

Use Case	I/O Bound?	Recommended Pool Size	Notes
Email notifications	Yes (SMTP/SES)	5–50 threads	Queue 1000; CallerRunsPolicy
PDF report generation	CPU-bound	= CPU cores	Queue 0 or small; avoid context switch
DB batch jobs	Yes (JDBC)	= HikariCP pool size	Avoid exceeding DB connections
External API calls	Yes (HTTP)	Virtual threads (Java 21)	No pool needed; VTs handle I/O

13-06 Spring WebFlux – Reactive

```
// Reactive product search – non-blocking pipeline
@RestController @RequestMapping("/reactive/v1")
public class ReactiveProductController {
private final ReactiveProductRepository repo;
@GetMapping("/products/{id}")
public Mono<ProductDto> getProduct(@PathVariable Long id) {
return repo.findById(id)
.switchIfEmpty(Mono.error(new ProductNotFoundException(id)))
.map(ProductMapper::toDto);
}
@GetMapping(value="/products/stream", produces=TEXT_EVENT_STREAM_VALUE)
public Flux<ProductDto> streamProducts() {
return repo.findAll()
.delayElements(Duration.ofMillis(100));
}
}
```

	Spring MVC (Servlet)	Spring WebFlux (Reactive)
Thread model	1 thread per request (blocking)	Event loop; non-blocking I/O
Max concurrent requests	~200 (thread pool)	Thousands (few event loop threads)
Backpressure	No native support	Built-in via Project Reactor
Learning curve	Low (familiar)	High (reactive paradigm)
Best for	CRUD; standard REST	Streaming; high concurrency; event-driven

13-07 Virtual Threads – Project Loom

Scenario	Platform Threads	Virtual Threads
10 000 concurrent HTTP requests	OOM – stack overflow at ~1k–10k threads	Trivial – 10k VTs use few carrier threads
JDBC query (blocking)	Blocks 1 OS thread per query	VT unmounts; carrier thread free for other VTs
Redis GET (blocking Jedis)	Blocks OS thread	VT unmounts on network I/O
CPU-bound computation	Native performance	No improvement – VT needs carrier thread

```
# Enable for all Spring MVC request handling
spring.threads.virtual.enabled=true # Spring Boot 3.2+
# Or configure Tomcat explicitly
@Bean TomcatProtocolHandlerCustomizer<?> virtualThreadCustomizer() {
return h -> h.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
}
# Benchmark: 10k concurrent requests
# Platform threads (200 pool): avg latency 1200ms; 8% error rate
# Virtual threads: avg latency 140ms; 0% error rate
```

13-08 **Kafka Async Event Processing**

Decoupling Write Path with Kafka

HTTP POST /orders – synchronous write path (fast path)

- 1. Validate order input & check stock (optimistic lock)
  - In-memory stock check (Redis cache) – sub-millisecond
- 2. Persist order to Postgres – commit
- 3. Publish OrderPlacedEvent to Kafka topic: orders.placed
- 4. Return 201 Created immediately (do NOT wait for async tasks)

Kafka consumers – async tasks (run after HTTP response returned)

- EmailConsumer – send order confirmation email
- InventoryConsumer – decrement DB stock (durable)
- LoyaltyConsumer – add loyalty points
- AnalyticsConsumer – update sales dashboard in Elasticsearch

Kafka Setting	Value	Purpose
acks=all	Producer	All replicas must ack – no data loss
enable.idempotence=true	Producer	Exactly-once delivery per partition
max.in.flight.requests.per.connection=1	Producer	Preserve ordering with idempotence
auto.offset.reset=earliest	Consumer	Don't miss events after restart

13-09 Bulk Operations

```
// JDBC batch insert – 10 000 rows
@Transactional
public void bulkInsertProducts(List<Product> products) {
jdbcTemplate.batchUpdate(
"INSERT INTO products(name,price,category_id,stock) VALUES(?,?,?,?)",
products, 500, // batch size = 500
(ps, p) -> {
ps.setString(1, p.getName());
ps.setBigDecimal(2, p.getPrice());
ps.setLong(3, p.getCategoryId());
ps.setInt(4, p.getStock());
});
}
// JPA bulk update (avoid loading entities into memory)
@Modifying @Transactional
@Query("UPDATE Product p SET p.price = p.price * :factor WHERE p.category.id = :catId")
int applyDiscount(@Param("factor") double factor, @Param("catId") Long catId);
```

Approach	Records/sec	Memory	Use When
JPA save() in loop	~500	High – entities in 1st cache	Never for bulk
saveAll() (batch)	~5 000	Medium – configured batch size	Up to 10K records
jdbcTemplate.batchUpdate	~50 000	Low – no entity lifecycle	Large CSV imports
COPY FROM (PostgreSQL native)	~500 000	Minimal	Initial data load; ETL

13-10 Pagination – Keyset vs Offset

Strategy	SQL	Performance	Consistent?
Offset pagination	LIMIT n OFFSET k	O(n+k) – slow for large k	No – inserts shift rows
Keyset pagination	WHERE id > lastId LIMIT n	O(log n) – index seek	Yes – stable cursor

```
// Keyset pagination – ShopVerse product listing
@Query("SELECT p FROM Product p WHERE p.id > :lastId ORDER BY p.id ASC")
Slice<Product> findNextPage(@Param("lastId") Long lastId, Pageable p);
// Response includes next cursor
{ "data": [...], "nextCursor": 4523, "hasMore": true }
```

Page vs Slice

	Page<T>	Slice<T>
Total count query	Yes (COUNT(*)) – expensive	No – just checks hasNext()
Use when	Need total pages/results	Infinite scroll; mobile feed
Performance	O(n) count + O(log n) data	O(log n) data only

13-11 Response Compression & Jackson

```
# Enable gzip compression in Spring Boot
server.compression.enabled=true
server.compression.mime-types=application/json,text/html
server.compression.min-response-size=1024
// Jackson performance tuning
@Bean public Jackson2ObjectMapperBuilderCustomizer jacksonCustomizer() {
return b -> b
.featuresToDisable(MapperFeature.DEFAULT_VIEW_INCLUSION)
.featuresToDisable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)
.serializationInclusion(JsonInclude.Include.NON_NULL) // skip null fields
.modules(new JavaTimeModule());
}
```

Optimisation	Before	After	Notes
gzip compression	100 KB JSON response	20 KB compressed	~80% reduction typical for JSON
NON_NULL serialisation	Nulls in JSON body	Nulls omitted	Smaller payload; cleaner API
@JsonView projection	Full DTO (20 fields)	List view (5 fields)	Avoid sending unused data
DTO projection (interface)	JPA loads entity + lazy	SELECT only needed columns	Fastest; no entity in memory

13-12 Load Testing with Gatling

```
class ProductLoadSimulation extends Simulation {
val httpConf = http.baseUrl("http://localhost:8080")
.acceptHeader("application/json");
val searchScenario = scenario("Product Search")
.exec(http("search_request")
.get("/api/v1/products/search?q=headphones&limit;=20")
.check(status.is(200))
.check(responseTimeInMillis.lt(500)));
setUp(
searchScenario.inject(
rampUsers(1000).during(60), // ramp to 1000 users over 60s
constantUsersPerSec(200).during(120) // sustain 200 rps for 2min
)
).protocols(httpConf)
.assertions(
global.responseTime.percentile(95).lt(500), // P95 < 500ms
global.failedRequests.percent.lt(1)); // < 1% errors
}
```

Metric	Target	Alert Threshold
P50 response time	< 100ms	>200ms
P95 response time	< 500ms	>1 000ms
P99 response time	< 1 000ms	>2 000ms
Error rate	< 0.1%	>1%
Throughput	200 RPS steady state	Drop >10% from baseline

BP Best Practices & Anti-Patterns

■ Do

- ✓ Enable virtual threads for I/O-bound Spring MVC apps – eliminates thread starvation at scale.
- ✓ Use EXPLAIN ANALYZE on slow queries; add composite indexes on (category_id, price, ratings_avg).
- ✓ Decouple slow async tasks (email, loyalty points) via Kafka – keeps HTTP response fast.
- ✓ Set connection-timeout=3000 on HikariCP – fail fast rather than queue threads indefinitely.
- ✓ Use keyset (cursor) pagination for large datasets – O(log n) vs O(n+k) for offset.
- ✓ Enable gzip compression for JSON responses over 1 KB – typical 70-80% size reduction.
- ✓ Use jdbcTemplate.batchUpdate for bulk inserts – 100× faster than JPA save() in loop.
- ✓ Set Cache-Control headers on static / infrequently-changed API endpoints (categories, config).
- ✓ Profile with async-profiler or VisualVM before optimising – measure, don't guess.
- ✓ Run Gatling load tests on every release; assert P95 < 500ms and error rate < 1%.

■ Anti-Patterns

- ■ SELECT * queries – load all columns even when only 2-3 are needed; use DTO projections.
- ■ JPA save() in a loop for bulk data – triggers one INSERT per row; use batchUpdate.
- ■ Offset pagination on large tables – OFFSET 100000 does a full scan before skipping.
- ■ Synchronous email send in order HTTP handler – email SMTP latency (200-500ms) in response.
- ■ Thread pool too large – more threads than CPU cores for CPU-bound tasks = context-switch overhead.
- ■ No connection pool (new connection per request) – TCP handshake + TLS + auth = 50-100ms overhead.
- ■ Ignoring GC logs in production – GC pauses cause latency spikes invisible in app metrics.

QR Quick Reference

Sub-Chapter	Key Point	One-Liner
13-01 JVM	ZGC for low pause	–XX:+UseZGC; dump heap on OOM; log GC
13-02 N+1	JOIN FETCH or @EntityGraph	EXPLAIN ANALYZE before adding indexes
13-03 HikariCP	Pool size = CPU*2 (SSD)	leak-detection-threshold=10s; timeout=3s
13-04 HTTP Cache	Cache-Control + ETag	304 Not Modified saves bandwidth & server load
13-05 @Async	CallerRunsPolicy	Queue fills before new threads; VTs for I/O
13-06 WebFlux	Mono/Flux non-blocking	Best for streaming or very high concurrency
13-07 Virtual Threads	spring.threads.virtual=true	I/O-bound throughput scales to millions
13-08 Kafka	Publish-then-respond	Never await Kafka in HTTP handler response path
13-09 Bulk	batchUpdate batch=500	COPY FROM for initial load; 500k rows/sec
13-10 Pagination	Keyset cursor pagination	WHERE id > lastId – stable & O(log n)
13-11 Compression	gzip min 1 KB	NON_NULL Jackson – omit null fields in JSON
13-12 Gatling	P95 < 500ms; errors < 1%	rampUsers then constantUsersPerSec assertions

Interview Quick-Check

Question	Concise Answer
How to fix N+1?	Add JOIN FETCH in JPQL query or @EntityGraph on repository method. Verify with Hibernate statis
HikariCP pool size?	Formula: CPU cores × 2 (for SSD/flash storage). Too large hurts DB; too small queues requests. Se
Kafka for async?	Publish event to Kafka after DB commit; return HTTP 201 immediately. Consumers handle email, inv
Keyset vs offset pagination?	Offset: OFFSET k scans k rows then skips – O(k). Keyset: WHERE id > cursor uses index – O(log n
Virtual thread benefit?	I/O-bound: VT unmounts from carrier on blocking I/O; carrier picks up next VT. 10k concurrent reque